

SQL Learning Material

From Scratch to Associate Software Engineer

Java Full Stack | L1 & L2 Interview Preparation

Target Role	Associate Software Engineer (Java Full Stack)
Interview Levels	L1 (Technical Screening) and L2 (Technical Deep-dive)
Prerequisites	None — starts from absolute basics
Coverage	DDL, DML, DQL, Joins, Subqueries, Indexes, Transactions,
	Normalization, Window Functions, CTEs, JDBC, Performance
Includes	100+ examples, 60+ interview Q&A, practice problems

Table of Contents

1. Introduction to Databases and SQL
2. RDBMS Concepts and Database Architecture
3. SQL Data Types
4. DDL Commands — CREATE, ALTER, DROP, TRUNCATE
5. DML Commands — INSERT, UPDATE, DELETE
6. DQL — SELECT Statement Fundamentals
7. Operators and Filtering (WHERE, BETWEEN, IN, LIKE)
8. Constraints — Primary Key, Foreign Key, Unique, Check
9. Built-in Functions (String, Numeric, Date)
10. Aggregate Functions, GROUP BY, HAVING
11. Joins — INNER, LEFT, RIGHT, FULL, CROSS, SELF
12. Subqueries — Scalar, Correlated, Nested
13. Set Operations — UNION, INTERSECT, EXCEPT
14. Views
15. Indexes and Query Performance
16. Stored Procedures and User-Defined Functions
17. Triggers
18. Transactions and ACID Properties
19. Normalization (1NF, 2NF, 3NF, BCNF)
20. Window Functions (ROW_NUMBER, RANK, LAG, LEAD)
21. Common Table Expressions (CTEs) and Recursive Queries
22. SQL with Java — JDBC, Connection Pooling, JPA Basics
23. Top 60+ Interview Questions with Answers
24. Practice Problems (Coding Round)
25. Final Tips, Tricky Concepts and Cheat Sheet

1. Introduction to Databases and SQL

What is a Database?

A **database** is an organized collection of structured information (data) stored electronically in a computer system. Databases let you store, modify, retrieve, and manage data efficiently. Without a database, applications would have to rely on files, which is slow, error-prone, and unsuitable for concurrent access.

DBMS vs RDBMS

DBMS (Database Management System): Software used to create and manage databases. Examples include older flat-file systems and hierarchical/network DBMS.

RDBMS (Relational DBMS): A DBMS based on the relational model where data is stored in tables (relations) with rows and columns, and tables can be related to each other through keys. Examples: **MySQL, PostgreSQL, Oracle, Microsoft SQL Server, IBM DB2.**

What is SQL?

SQL (Structured Query Language) is the standard language used to communicate with relational databases. It is declarative — you describe *what* you want, not *how* to get it. SQL was standardized by ANSI in 1986 and ISO in 1987.

Categories of SQL Commands

Category	Full Form	Commands	Purpose
DDL	Data Definition Language	CREATE, ALTER, DROP, TRUNCATE, RENAME	Define schema
DML	Data Manipulation Language	INSERT, UPDATE, DELETE	Modify data
DQL	Data Query Language	SELECT	Retrieve data
DCL	Data Control Language	GRANT, REVOKE	Permissions
TCL	Transaction Control Language	COMMIT, ROLLBACK, SAVEPOINT	Manage transactions

Note: Interviewers love asking the difference between **TRUNCATE**, **DELETE**, and **DROP**. We will cover this in Section 4.

2. RDBMS Concepts and Database Architecture

Key Terminology

- **Table (Relation):** A collection of rows and columns.
- **Row (Tuple / Record):** A single entry in a table.
- **Column (Attribute / Field):** A named property with a specific data type.
- **Schema:** The structure (blueprint) of a database — tables, columns, types, constraints.
- **Instance:** The actual data in the database at a given moment.
- **Domain:** The set of valid values for an attribute.
- **Degree:** Number of columns in a table.
- **Cardinality:** Number of rows in a table.

Types of Keys

- **Primary Key:** Uniquely identifies each row. Cannot be NULL. Only one per table.
- **Candidate Key:** Any column (or set) that *could* serve as primary key.
- **Super Key:** A set of columns that uniquely identifies rows (may include extras).
- **Alternate Key:** Candidate keys not chosen as the primary key.
- **Foreign Key:** A column referring to the primary key of another table — enforces referential integrity.
- **Composite Key:** A primary key made of two or more columns.
- **Unique Key:** Ensures uniqueness but allows one NULL.
- **Surrogate Key:** An artificial key (e.g., auto-increment ID) with no business meaning.

Relationships in RDBMS

- **One-to-One (1:1):** One row in A relates to one row in B (e.g., User ↔ Passport).
- **One-to-Many (1:N):** One row in A relates to many in B (e.g., Customer ↔ Orders).
- **Many-to-Many (M:N):** Requires a junction/bridge table (e.g., Students ↔ Courses).

3. SQL Data Types

Category	Type	Description	Example
Numeric	INT / INTEGER	Whole numbers	42
	BIGINT	Large whole numbers	9999999999
	DECIMAL(p,s) / NUMERIC	Exact decimals	DECIMAL(10,2) → 12345.67
	FLOAT / DOUBLE	Approximate decimals	3.14159
String	CHAR(n)	Fixed length	CHAR(5) — 'Hi '
	VARCHAR(n)	Variable length	'Hello'
	TEXT	Long text	Articles, descriptions
Date/Time	DATE	Date only	2025-06-13
	TIME	Time only	14:30:00
	DATETIME / TIMESTAMP	Date + Time	2025-06-13 14:30:00
Boolean	BOOLEAN / BIT	TRUE / FALSE	TRUE
Binary	BLOB / VARBINARY	Binary data	Images, files

CHAR vs VARCHAR: CHAR pads with spaces to a fixed length; VARCHAR stores only the actual content plus 1-2 bytes for length. Use CHAR for fixed-size codes (e.g., country code 'IN'), VARCHAR for names, emails.

4. DDL Commands — CREATE, ALTER, DROP, TRUNCATE

CREATE TABLE

```
CREATE TABLE employees (  
  emp_id      INT PRIMARY KEY AUTO_INCREMENT,  
  first_name  VARCHAR(50) NOT NULL,  
  last_name   VARCHAR(50) NOT NULL,  
  email       VARCHAR(100) UNIQUE,  
  salary      DECIMAL(10, 2) DEFAULT 0.00,  
  dept_id     INT,  
  hire_date   DATE DEFAULT CURRENT_DATE,  
  FOREIGN KEY (dept_id) REFERENCES departments(dept_id)  
);
```

ALTER TABLE

```
-- Add a column  
ALTER TABLE employees ADD phone VARCHAR(15);  
  
-- Modify a column type  
ALTER TABLE employees MODIFY salary DECIMAL(12, 2);  
  
-- Rename a column (MySQL 8+)  
ALTER TABLE employees RENAME COLUMN phone TO mobile;  
  
-- Drop a column  
ALTER TABLE employees DROP COLUMN mobile;  
  
-- Add a constraint  
ALTER TABLE employees ADD CONSTRAINT chk_salary CHECK (salary >= 0);
```

DROP vs TRUNCATE vs DELETE

Aspect	DROP	TRUNCATE	DELETE
Category	DDL	DDL	DML
Removes	Entire table + structure	All rows, keeps structure	Rows (filterable)
WHERE clause	No	No	Yes
Rollback	No (auto-commit)	No (in most DBs)	Yes (within TXN)
Speed	Fast	Very fast	Slower (logs each row)
Triggers fire?	No	No	Yes
Resets auto-increment?	N/A (table gone)	Yes	No

```
DROP TABLE employees;           -- Removes table completely  
TRUNCATE TABLE employees;       -- Empties table, keeps structure  
DELETE FROM employees WHERE id = 5; -- Removes specific row(s)
```

5. DML Commands — INSERT, UPDATE, DELETE

INSERT

```
-- Single row
INSERT INTO employees (first_name, last_name, email, salary, dept_id)
VALUES ('Arun', 'Kumar', 'arun@example.com', 55000.00, 1);

-- Multiple rows
INSERT INTO employees (first_name, last_name, email, salary, dept_id)
VALUES
    ('Priya', 'Sharma', 'priya@example.com', 62000, 2),
    ('Ravi', 'Patel', 'ravi@example.com', 48000, 1),
    ('Meera', 'Iyer', 'meera@example.com', 71000, 3);

-- Insert from another table
INSERT INTO ex_employees (emp_id, full_name, salary)
SELECT emp_id, CONCAT(first_name, ' ', last_name), salary
FROM employees WHERE status = 'INACTIVE';
```

UPDATE

```
-- Update one column
UPDATE employees SET salary = 60000 WHERE emp_id = 1;

-- Update multiple columns
UPDATE employees
SET salary = salary * 1.10,
    last_modified = NOW()
WHERE dept_id = 2;

-- Update with JOIN (MySQL syntax)
UPDATE employees e
JOIN departments d ON e.dept_id = d.dept_id
SET e.salary = e.salary * 1.15
WHERE d.dept_name = 'Engineering';
```

DELETE

```
DELETE FROM employees WHERE emp_id = 5;
DELETE FROM employees WHERE salary < 30000;
DELETE FROM employees; -- Deletes ALL rows
-- ALWAYS use WHERE unless you really mean to delete all rows!
```

6. DQL — SELECT Statement Fundamentals

Basic SELECT Structure

```
SELECT [DISTINCT] column1, column2, ...
FROM   table_name
[WHERE condition]
[GROUP BY column(s)]
[HAVING condition]
[ORDER BY column(s) [ASC | DESC]]
[LIMIT n [OFFSET m]];
```

Logical Order of Execution (very common interview question)

```
1. FROM      -- pick tables
2. JOIN      -- combine tables
3. WHERE     -- filter rows
4. GROUP BY  -- group rows
5. HAVING    -- filter groups
6. SELECT    -- pick columns
7. DISTINCT  -- remove duplicates
8. ORDER BY  -- sort
9. LIMIT     -- restrict rows
```

Common SELECT Examples

```
-- All columns, all rows
SELECT * FROM employees;

-- Specific columns
SELECT first_name, salary FROM employees;

-- Distinct values
SELECT DISTINCT dept_id FROM employees;

-- Aliases
SELECT first_name AS fname, salary * 12 AS annual_salary FROM employees;

-- Sorting
SELECT * FROM employees ORDER BY salary DESC, hire_date ASC;

-- Pagination
SELECT * FROM employees ORDER BY emp_id LIMIT 10 OFFSET 20; -- Page 3 of size 10
```


7. Operators and Filtering (WHERE, BETWEEN, IN, LIKE)

Comparison & Logical Operators

- **Comparison:** = , != or <> , > , < , >= , <=
- **Logical:** AND, OR, NOT
- **Range:** BETWEEN a AND b (inclusive on both ends)
- **List:** IN (v1, v2, ...) and NOT IN
- **Pattern:** LIKE — % matches any sequence, _ matches exactly one character
- **NULL check:** IS NULL, IS NOT NULL (never use = NULL)

Examples

```
-- BETWEEN
SELECT * FROM employees WHERE salary BETWEEN 40000 AND 70000;

-- IN
SELECT * FROM employees WHERE dept_id IN (1, 2, 5);

-- LIKE
SELECT * FROM employees WHERE first_name LIKE 'A%';      -- starts with A
SELECT * FROM employees WHERE email LIKE '%@gmail.com'; -- ends with
SELECT * FROM employees WHERE first_name LIKE '_ravi';   -- 5 chars ending 'ravi'

-- NULL handling
SELECT * FROM employees WHERE manager_id IS NULL;

-- Combining conditions
SELECT * FROM employees
WHERE dept_id = 2 AND (salary > 60000 OR hire_date < '2022-01-01');
```

Why = NULL fails: In SQL, NULL means 'unknown'. Any comparison with NULL using = returns UNKNOWN (not TRUE), so the row is excluded. Always use IS NULL / IS NOT NULL.

8. Constraints

Constraints enforce rules on data to maintain accuracy and integrity. They can be applied at column-level or table-level, during CREATE TABLE or added later with ALTER TABLE.

Types of Constraints

- **NOT NULL:** Column must have a value.
- **UNIQUE:** All values must be distinct (allows one NULL).
- **PRIMARY KEY:** NOT NULL + UNIQUE. Only one per table.
- **FOREIGN KEY:** Enforces a link to another table's primary key.
- **CHECK:** Ensures a condition is true (e.g., salary >= 0).
- **DEFAULT:** Provides a default if no value is given.

```
CREATE TABLE students (  
    student_id INT PRIMARY KEY,  
    name       VARCHAR(50) NOT NULL,  
    email      VARCHAR(100) UNIQUE,  
    age        INT CHECK (age >= 17),  
    grade      CHAR(1) DEFAULT 'F',  
    course_id  INT,  
    CONSTRAINT fk_course FOREIGN KEY (course_id)  
        REFERENCES courses(course_id)  
        ON DELETE CASCADE  
        ON UPDATE CASCADE  
);
```

Referential Actions (ON DELETE / ON UPDATE)

- **CASCADE:** Propagate the change to child rows.
- **SET NULL:** Set the foreign key column to NULL.
- **SET DEFAULT:** Set to the default value.
- **RESTRICT / NO ACTION:** Reject the change if dependent rows exist.

9. Built-in Functions

String Functions

```
LENGTH('Hello')           -- 5
UPPER('hello')             -- 'HELLO'
LOWER('HELLO')             -- 'hello'
CONCAT('John', ' ', 'Doe') -- 'John Doe'
SUBSTRING('Database', 1, 4) -- 'Data'
TRIM('  hi  ')             -- 'hi'
REPLACE('abcabc', 'a', 'X') -- 'XbcXbc'
INSTR('hello', 'll')       -- 3    (position)
LEFT('database', 4)        -- 'data'
RIGHT('database', 4)       -- 'base'
```

Numeric Functions

```
ABS(-15)      -- 15
ROUND(3.756, 1) -- 3.8
CEIL(4.2)     -- 5
FLOOR(4.9)    -- 4
MOD(10, 3)    -- 1
POWER(2, 10)  -- 1024
SQRT(16)     -- 4
```

Date Functions (MySQL)

```
NOW()           -- current date+time
CURDATE()       -- current date
CURTIME()       -- current time
DATE_ADD(NOW(), INTERVAL 7 DAY) -- 7 days from now
DATEDIFF('2025-12-31', '2025-06-13') -- 201
YEAR(hire_date) -- 2024
MONTH(hire_date) -- 7
DATE_FORMAT(NOW(), '%d-%m-%Y') -- '13-06-2026'
```

NULL-handling Functions

```
IFNULL(salary, 0)           -- replace NULL with 0
COALESCE(phone, mobile, 'N/A') -- first non-NULL
NULLIF(a, b)                -- NULL if a = b, else a
CASE WHEN salary IS NULL THEN 0 ELSE salary END
```

10. Aggregate Functions, GROUP BY, HAVING

Aggregate Functions

- **COUNT(*)** — counts all rows including NULLs
- **COUNT(col)** — counts non-NULL values in col
- **COUNT(DISTINCT col)** — counts unique non-NULL values
- **SUM(col), AVG(col), MIN(col), MAX(col)**

```
SELECT COUNT(*)          FROM employees;
SELECT AVG(salary)       FROM employees;
SELECT MAX(salary)       FROM employees WHERE dept_id = 1;
SELECT COUNT(DISTINCT dept_id) FROM employees;
```

GROUP BY

```
-- Total salary per department
SELECT dept_id, SUM(salary) AS total_salary, COUNT(*) AS headcount
FROM employees
GROUP BY dept_id;

-- Multiple grouping columns
SELECT dept_id, gender, AVG(salary)
FROM employees
GROUP BY dept_id, gender;
```

HAVING vs WHERE

WHERE filters rows *before* grouping. **HAVING** filters groups *after* aggregation. You can use aggregate functions in HAVING but not in WHERE.

```
-- Departments with avg salary > 50000
SELECT dept_id, AVG(salary) AS avg_sal
FROM employees
WHERE hire_date >= '2020-01-01' -- filter rows first
GROUP BY dept_id
HAVING AVG(salary) > 50000      -- filter groups
ORDER BY avg_sal DESC;
```

11. Joins

A **JOIN** combines rows from two or more tables based on a related column. JOINS are one of the most heavily tested topics — expect to be asked all the time.

Sample Tables

```
employees(emp_id, name, dept_id)
departments(dept_id, dept_name)
```

INNER JOIN

Returns only matching rows from both tables.

```
SELECT e.name, d.dept_name
FROM employees e
INNER JOIN departments d ON e.dept_id = d.dept_id;
```

LEFT (OUTER) JOIN

All rows from the LEFT table, with matching rows from the right (NULL if no match).

```
SELECT e.name, d.dept_name
FROM employees e
LEFT JOIN departments d ON e.dept_id = d.dept_id;
-- Employees without a department still appear (dept_name will be NULL).
```

RIGHT (OUTER) JOIN

All rows from the RIGHT table, with matching rows from the left.

```
SELECT e.name, d.dept_name
FROM employees e
RIGHT JOIN departments d ON e.dept_id = d.dept_id;
-- Departments without employees still appear.
```

FULL (OUTER) JOIN

Returns all rows from both tables. MySQL doesn't support FULL JOIN directly — simulate with UNION:

```
-- Standard SQL
SELECT e.name, d.dept_name
FROM employees e
FULL OUTER JOIN departments d ON e.dept_id = d.dept_id;

-- MySQL workaround
SELECT e.name, d.dept_name
FROM employees e LEFT JOIN departments d ON e.dept_id = d.dept_id
UNION
SELECT e.name, d.dept_name
FROM employees e RIGHT JOIN departments d ON e.dept_id = d.dept_id;
```

CROSS JOIN

Cartesian product — every row of A combined with every row of B.

```
SELECT a.color, b.size FROM colors a CROSS JOIN sizes b;
-- If colors has 3 rows and sizes has 4, the result has 12 rows.
```

SELF JOIN

Joining a table with itself — useful for hierarchical data.

```
-- Find each employee and their manager's name
SELECT e.name AS employee, m.name AS manager
FROM employees e
LEFT JOIN employees m ON e.manager_id = m.emp_id;
```

Quick Visual Cheat-sheet

Join Type	What it returns
INNER JOIN	Only matching rows from both
LEFT JOIN	All from left + matching from right (NULL if none)
RIGHT JOIN	All from right + matching from left
FULL OUTER	All rows from both (NULLs where no match)
CROSS JOIN	Cartesian product ($A \times B$)
SELF JOIN	Table joined with itself

12. Subqueries — Scalar, Correlated, Nested

A **subquery** is a query nested inside another query. It can appear in SELECT, FROM, WHERE, or HAVING clauses.

Single-row (Scalar) Subquery

```
-- Employees earning more than the average
SELECT name, salary
FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);
```

Multi-row Subquery (IN / ANY / ALL)

```
SELECT name FROM employees
WHERE dept_id IN (SELECT dept_id FROM departments WHERE location = 'Chennai');

-- Salary greater than ALL salaries in dept 2
SELECT name FROM employees
WHERE salary > ALL (SELECT salary FROM employees WHERE dept_id = 2);

-- Salary greater than ANY (i.e., greater than minimum) in dept 2
SELECT name FROM employees
WHERE salary > ANY (SELECT salary FROM employees WHERE dept_id = 2);
```

Correlated Subquery

A correlated subquery references a column from the outer query. It runs once per outer row (conceptually) so it can be slower than a normal subquery.

```
-- Employees whose salary is greater than the average of their own department
SELECT e1.name, e1.salary, e1.dept_id
FROM employees e1
WHERE e1.salary > (
    SELECT AVG(e2.salary)
    FROM employees e2
    WHERE e2.dept_id = e1.dept_id
);
```

EXISTS vs IN

```
-- EXISTS: returns true as soon as the subquery returns any row
SELECT name FROM employees e
WHERE EXISTS (
    SELECT 1 FROM departments d
    WHERE d.dept_id = e.dept_id AND d.location = 'Chennai'
);

-- IN: compares the value to a list. NULLs in IN can be tricky.
SELECT name FROM employees
WHERE dept_id IN (SELECT dept_id FROM departments WHERE location = 'Chennai');
```

Classic: 2nd Highest Salary (multiple methods)

```
-- Method 1: subquery
SELECT MAX(salary) AS second_highest
FROM employees
WHERE salary < (SELECT MAX(salary) FROM employees);

-- Method 2: LIMIT
```

```
SELECT DISTINCT salary FROM employees
ORDER BY salary DESC LIMIT 1 OFFSET 1;

-- Method 3: DENSE_RANK (handles ties correctly)
SELECT salary FROM (
    SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) AS rk
    FROM employees
) t WHERE rk = 2;
```


13. Set Operations

- **UNION:** Combines results, removes duplicates.
- **UNION ALL:** Combines results, keeps duplicates (faster).
- **INTERSECT:** Rows common to both.
- **EXCEPT / MINUS:** Rows in first query but not in second.

Rules: All queries must have the same number of columns with compatible data types. Column names come from the first query.

```
SELECT name FROM customers
UNION
SELECT name FROM employees;
```

```
SELECT name FROM customers
UNION ALL
SELECT name FROM employees;
```

```
SELECT name FROM customers
INTERSECT
SELECT name FROM employees; -- not supported in MySQL until 8.0.31
```

14. Views

A **view** is a virtual table based on a SELECT query. It stores the query, not the data. Each time you access the view, the underlying query runs.

Benefits of Views

- **Security:** Expose only relevant columns.
- **Simplification:** Hide complex joins behind a simple name.
- **Reusability:** Avoid rewriting the same query.
- **Logical independence:** Change underlying schema without breaking apps.

```
CREATE VIEW v_high_earners AS
SELECT e.emp_id, e.name, d.dept_name, e.salary
FROM employees e JOIN departments d ON e.dept_id = d.dept_id
WHERE e.salary > 75000;

-- Use it like a table
SELECT * FROM v_high_earners WHERE dept_name = 'Engineering';

-- Drop / replace
DROP VIEW v_high_earners;
CREATE OR REPLACE VIEW v_high_earners AS ...;
```

Updatable views: A view is updatable only if it maps directly to a single base table without DISTINCT, GROUP BY, aggregates, UNION, etc. Materialized views (Oracle/PostgreSQL) actually store the result and need refreshing — MySQL doesn't support them natively.

15. Indexes and Query Performance

An **index** is a data structure (typically a B-tree) that speeds up data retrieval — like an index in a book. Indexes trade extra storage and slower writes for faster reads.

Types of Indexes

- **Single-column:** Index on one column.
- **Composite (Multi-column):** Index on two or more columns; left-most prefix rule matters.
- **Unique:** Enforces uniqueness in addition to speed.
- **Clustered:** Data rows are physically stored in index order (one per table, usually the PK).
- **Non-clustered:** Index points to row locations; many per table.
- **Full-text:** For text searching (MATCH ... AGAINST).
- **Hash:** Fast equality lookups (no range scans).

```
CREATE INDEX idx_emp_lastname ON employees(last_name);
CREATE UNIQUE INDEX idx_emp_email ON employees(email);
CREATE INDEX idx_emp_dept_salary ON employees(dept_id, salary);
DROP INDEX idx_emp_lastname ON employees;
```

```
-- Inspect query plan
EXPLAIN SELECT * FROM employees WHERE last_name = 'Kumar';
```

When NOT to Index

- Tables that are very small.
- Columns updated very frequently (index maintenance overhead).
- Columns with very few distinct values (low cardinality, e.g. boolean).
- Tables that are write-heavy and rarely queried by that column.

Common Performance Tips

- Avoid **SELECT ***; fetch only needed columns.
- Use indexes on JOIN and WHERE columns.
- Don't apply functions on indexed columns in WHERE — they break the index (e.g., *WHERE YEAR(hire_date) = 2024* bypasses index).
- Use LIMIT for pagination.
- Use EXPLAIN / EXPLAIN ANALYZE to verify the query plan.
- Prefer JOINS to correlated subqueries when possible.
- Batch INSERTs instead of one-at-a-time.
- Keep transactions short.

16. Stored Procedures and User-Defined Functions

Stored Procedure

A precompiled block of SQL stored in the database. Can have IN, OUT, INOUT parameters. Cannot be used inside a SELECT directly.

```
DELIMITER //
CREATE PROCEDURE GetEmployeesByDept(IN deptId INT)
BEGIN
    SELECT * FROM employees WHERE dept_id = deptId;
END //
DELIMITER ;

CALL GetEmployeesByDept(2);
```

User-Defined Function (UDF)

Returns a single value, usable in SELECT/WHERE. Cannot perform transactions or alter data (in most DBs).

```
DELIMITER //
CREATE FUNCTION GetAnnualSalary(monthlySalary DECIMAL(10,2))
RETURNS DECIMAL(12,2)
DETERMINISTIC
BEGIN
    RETURN monthlySalary * 12;
END //
DELIMITER ;

SELECT name, GetAnnualSalary(salary) AS yearly FROM employees;
```

Procedure vs Function

Aspect	Stored Procedure	Function
Return	Optional (via OUT params)	Must return one value
Usable in SELECT	No	Yes
DML allowed	Yes	Limited
Called by	CALL	Used inline in expressions
Transactions	Allowed	Usually not

17. Triggers

A **trigger** is SQL code that runs automatically in response to events (INSERT, UPDATE, DELETE) on a table. Triggers fire BEFORE or AFTER the event.

```
-- Audit trail for salary changes
CREATE TABLE salary_audit (
    audit_id    INT AUTO_INCREMENT PRIMARY KEY,
    emp_id      INT,
    old_salary  DECIMAL(10,2),
    new_salary  DECIMAL(10,2),
    changed_on  DATETIME
);

DELIMITER //
CREATE TRIGGER trg_salary_audit
AFTER UPDATE ON employees
FOR EACH ROW
BEGIN
    IF OLD.salary <> NEW.salary THEN
        INSERT INTO salary_audit(emp_id, old_salary, new_salary, changed_on)
        VALUES (OLD.emp_id, OLD.salary, NEW.salary, NOW());
    END IF;
END //
DELIMITER ;
```

OLD vs NEW

- **OLD:** The row before the change (UPDATE, DELETE).
- **NEW:** The row after the change (INSERT, UPDATE).

18. Transactions and ACID Properties

A **transaction** is a sequence of one or more SQL statements that execute as a single unit. Either all succeed (COMMIT) or none take effect (ROLLBACK).

ACID Properties (must-know)

Property	Meaning
Atomicity	All operations succeed, or none do (all-or-nothing).
Consistency	Database moves from one valid state to another, respecting constraints.
Isolation	Concurrent transactions don't interfere with each other.
Durability	Once committed, changes survive crashes/power failures.

TCL Commands

```
START TRANSACTION;  -- or BEGIN
UPDATE accounts SET balance = balance - 1000 WHERE id = 1;
UPDATE accounts SET balance = balance + 1000 WHERE id = 2;
SAVEPOINT sp1;
UPDATE accounts SET balance = balance - 500 WHERE id = 1;
ROLLBACK TO sp1;    -- undo only after sp1
COMMIT;             -- finalize
```

Isolation Levels (low → high)

Level	Dirty Read	Non-repeatable	Phantom Read
READ UNCOMMITTED	Yes	Yes	Yes
READ COMMITTED	No	Yes	Yes
REPEATABLE READ	No	No	Yes (Maybe)
SERIALIZABLE	No	No	No

Concurrency Problems Explained

- **Dirty Read:** Reading uncommitted data from another transaction.
- **Non-repeatable Read:** Same row read twice gives different values.
- **Phantom Read:** Same query returns different sets of rows.
- **Lost Update:** Two transactions overwrite each other's changes.

19. Normalization

Normalization organizes data to reduce redundancy and improve integrity. It splits tables based on functional dependencies.

1NF — First Normal Form

- Each column contains atomic (indivisible) values.
- Each column has values of the same type.
- Each row is unique (a primary key exists).
- No repeating groups or arrays.

Violation: A column 'phones' = '123, 456'. **Fix:** Split into separate rows or a phones table.

2NF — Second Normal Form

- Must be in 1NF.
- No **partial dependency** — non-key columns must depend on the *entire* composite key.

Example: If PK is (student_id, course_id), and student_name depends only on student_id, that's a partial dependency. **Fix:** Move student_name to a Students table.

3NF — Third Normal Form

- Must be in 2NF.
- No **transitive dependency** — non-key columns shouldn't depend on other non-key columns.

Example: employees(emp_id, dept_id, dept_name). dept_name depends on dept_id (not emp_id). **Fix:** Move dept_name to departments table.

BCNF — Boyce-Codd Normal Form

A stricter 3NF. For every functional dependency $X \rightarrow Y$, X must be a super key.

Denormalization

Intentionally introducing redundancy to improve **read performance** (fewer joins). Common in reporting/analytics tables. Trade-off: more storage and harder updates.

20. Window Functions

Window functions perform calculations across a set of rows related to the current row, without collapsing rows like GROUP BY does. Syntax: **function() OVER (PARTITION BY ... ORDER BY ...)**.

Ranking Functions

```
SELECT name, dept_id, salary,
       ROW_NUMBER() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS rn,
       RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS rk,
       DENSE_RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS drk
FROM employees;
```

Difference between ROW_NUMBER / RANK / DENSE_RANK

Salary	ROW_NUMBER	RANK	DENSE_RANK
90000	1	1	1
90000	2	1	1
80000	3	3	2
70000	4	4	3

LAG and LEAD

```
-- Compare a row with the previous and next rows
SELECT name, hire_date, salary,
       LAG(salary) OVER (ORDER BY hire_date) AS prev_salary,
       LEAD(salary) OVER (ORDER BY hire_date) AS next_salary
FROM employees;
```

Aggregate Window Functions

```
-- Running total of salaries
SELECT name, salary,
       SUM(salary) OVER (ORDER BY emp_id) AS running_total,
       AVG(salary) OVER (PARTITION BY dept_id) AS dept_avg
FROM employees;
```


21. Common Table Expressions (CTEs) and Recursive Queries

A **CTE** (Common Table Expression) is a temporary, named result set defined using the **WITH** clause. It exists only for the query and makes complex queries far more readable.

Basic CTE

```
WITH dept_stats AS (  
    SELECT dept_id, AVG(salary) AS avg_sal, COUNT(*) AS headcount  
    FROM employees  
    GROUP BY dept_id  
)  
SELECT d.dept_name, s.avg_sal, s.headcount  
FROM dept_stats s JOIN departments d ON s.dept_id = d.dept_id  
WHERE s.headcount > 5;
```

CTE vs Subquery

- CTEs are reusable inside the same query (can reference multiple times).
- Better readability for complex logic.
- Allow recursion — subqueries don't.

Recursive CTE

Used to walk hierarchical/tree data — like an org chart or a category tree.

```
-- Find all employees under a manager (recursive)  
WITH RECURSIVE org_tree AS (  
    -- Anchor: top manager  
    SELECT emp_id, name, manager_id, 1 AS level  
    FROM employees  
    WHERE manager_id IS NULL  
  
    UNION ALL  
  
    -- Recursive part  
    SELECT e.emp_id, e.name, e.manager_id, t.level + 1  
    FROM employees e  
    INNER JOIN org_tree t ON e.manager_id = t.emp_id  
)  
SELECT * FROM org_tree ORDER BY level, emp_id;
```

22. SQL with Java — JDBC, Connection Pooling, JPA Basics

As a Java full stack developer, you'll most often interact with SQL through **JDBC**, **Spring JDBC**, or **JPA/Hibernate**. Interviewers expect at least a working knowledge of JDBC.

JDBC: 7 Standard Steps

```
// 1. Load driver (optional in JDBC 4.0+)
Class.forName("com.mysql.cj.jdbc.Driver");

// 2. Establish connection
Connection conn = DriverManager.getConnection(
    "jdbc:mysql://localhost:3306/companydb", "root", "password");

// 3. Create statement (use PreparedStatement to avoid SQL injection!)
String sql = "SELECT emp_id, name, salary FROM employees WHERE dept_id = ?";
PreparedStatement ps = conn.prepareStatement(sql);

// 4. Bind parameters
ps.setInt(1, 2);

// 5. Execute query
ResultSet rs = ps.executeQuery();

// 6. Process results
while (rs.next()) {
    int id = rs.getInt("emp_id");
    String name = rs.getString("name");
    double sal = rs.getDouble("salary");
    System.out.printf("%d %s %.2f\n", id, name, sal);
}

// 7. Close resources (use try-with-resources in practice)
rs.close(); ps.close(); conn.close();
```

Statement vs PreparedStatement vs CallableStatement

Type	Purpose	SQL Injection Safe
Statement	Static SQL, one-time	No
PreparedStatement	Parameterized, reusable, faster	Yes
CallableStatement	Calls stored procedures	Yes

Transaction Management in JDBC

```
conn.setAutoCommit(false);
try {
    ps1.executeUpdate(); // debit
    ps2.executeUpdate(); // credit
    conn.commit();
} catch (SQLException e) {
    conn.rollback();
    throw e;
} finally {
    conn.setAutoCommit(true);
}
```

Connection Pooling

Opening a JDBC connection is expensive. A **connection pool** maintains a set of open connections and hands them out as needed. Popular pools: **HikariCP** (default in Spring Boot), Apache DBCP, c3p0.

JPA / Hibernate (one-paragraph primer)

JPA is a Java spec for ORM (Object-Relational Mapping). **Hibernate** is the most popular implementation. You annotate Java classes with `@Entity`, `@Id`, `@Column`, `@ManyToOne`, etc., and the framework generates SQL behind the scenes. Spring Data JPA further reduces boilerplate via repository interfaces.

```
@Entity
@Table(name = "employees")
public class Employee {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long empId;

    @Column(nullable = false)
    private String name;

    @ManyToOne
    @JoinColumn(name = "dept_id")
    private Department department;
    // getters/setters
}

public interface EmployeeRepository extends JpaRepository<Employee, Long> {
    List<Employee> findByDepartment_DeptName(String deptName);
}
```

SQL Injection (a must-know vulnerability)

```
// VULNERABLE – string concatenation
String sql = "SELECT * FROM users WHERE email = '" + email + "'";
// Attacker can pass: ' OR '1'='1
// resulting in:      SELECT * FROM users WHERE email = '' OR '1'='1'

// SAFE – parameterized
PreparedStatement ps = conn.prepareStatement(
    "SELECT * FROM users WHERE email = ?");
ps.setString(1, email);
```

23. Top 60+ Interview Questions with Answers

L1 — Fundamentals

Q1. What is the difference between SQL and MySQL?

SQL is a query language used to interact with relational databases. MySQL is an RDBMS that uses SQL as its language. SQL is the standard; MySQL is one specific product (others: PostgreSQL, Oracle, SQL Server).

Q2. What are the different types of SQL commands?

DDL (CREATE, ALTER, DROP, TRUNCATE), DML (INSERT, UPDATE, DELETE), DQL (SELECT), DCL (GRANT, REVOKE), TCL (COMMIT, ROLLBACK, SAVEPOINT).

Q3. Difference between DELETE, TRUNCATE, and DROP?

DELETE removes rows one at a time (WHERE allowed, can be rolled back, triggers fire). TRUNCATE removes all rows quickly (no WHERE, resets auto-increment, no triggers). DROP removes the entire table including structure.

Q4. What is a primary key vs a unique key?

PRIMARY KEY: uniquely identifies a row, NOT NULL, only one per table. UNIQUE KEY: ensures uniqueness, allows one NULL, can be multiple per table.

Q5. What is a foreign key?

A column (or set) in one table that references the primary key of another table to enforce referential integrity. It prevents orphan records.

Q6. What is the difference between CHAR and VARCHAR?

CHAR is fixed-length (pads with spaces). VARCHAR is variable-length (uses only the actual length + 1-2 bytes for length). Use CHAR for short, fixed codes; VARCHAR for general text.

Q7. What is the difference between WHERE and HAVING?

WHERE filters individual rows before grouping; cannot use aggregate functions. HAVING filters groups after GROUP BY; can use aggregate functions.

Q8. What does the GROUP BY clause do?

Groups rows that share the same values in specified columns so aggregate functions can compute summary values per group.

Q9. What is the difference between UNION and UNION ALL?

UNION removes duplicates (slower). UNION ALL keeps all rows including duplicates (faster). Always prefer UNION ALL when you know there are no duplicates.

Q10. What is a JOIN? Name the types.

A JOIN combines rows from two or more tables based on a related column. Types: INNER, LEFT, RIGHT, FULL OUTER, CROSS, SELF.

Q11. What is a self join?

A join of a table with itself, useful for hierarchical data (e.g., finding each employee's manager when both are in the same employees table).

Q12. How is NULL different from 0 or an empty string?

NULL represents 'unknown' or 'missing'. 0 is a numeric value; "" is an empty string. NULL is not equal to anything — even another NULL. Use IS NULL to test for it.

Q13. What is a subquery?

A query nested inside another query. Can be scalar (returns one value), single-row, multi-row, or correlated (references the outer query).

Q14. What is a correlated subquery?

A subquery that uses a value from the outer query. It is evaluated once for each row processed by the outer query, which can make it slower than a non-correlated subquery.

Q15. What is the difference between IN and EXISTS?

IN compares a column to a list/result. EXISTS returns true as soon as the subquery returns any row. EXISTS often performs better with correlated subqueries and handles NULLs more predictably.

Q16. How do you find the 2nd highest salary?

Three ways: (1) `SELECT MAX(salary) FROM employees WHERE salary < (SELECT MAX(salary) FROM employees)`; (2) `ORDER BY salary DESC LIMIT 1 OFFSET 1`; (3) Window function `DENSE_RANK()`.

Q17. What is normalization? List the normal forms.

Process of organizing data to reduce redundancy. Common forms: 1NF (atomic columns), 2NF (no partial dependency), 3NF (no transitive dependency), BCNF (every determinant is a super key).

Q18. What is denormalization?

Intentionally introducing redundancy (e.g., duplicating columns or pre-joining tables) to speed up reads at the cost of update complexity and storage.

Q19. What is an index? Why use one?

A data structure (typically B-tree) that speeds up lookups by avoiding full table scans. Improves SELECT performance but slows INSERT/UPDATE/DELETE.

Q20. Difference between clustered and non-clustered indexes?

Clustered: data is physically sorted by the index — only one per table (usually the PK).
Non-clustered: index has pointers to row locations — many per table.

Q21. What is a view?

A virtual table defined by a SELECT query. It doesn't store data (except materialized views). Used for security, simplification, and abstraction.

Q22. Can we update data through a view?

Yes, but only if the view maps to a single base table and doesn't include DISTINCT, GROUP BY, aggregates, UNION, joins, or computed columns.

L2 — Advanced

Q23. Explain ACID properties.

Atomicity: all-or-nothing. Consistency: DB moves from one valid state to another. Isolation: concurrent transactions don't interfere. Durability: committed changes survive failures.

Q24. What are transaction isolation levels?

READ UNCOMMITTED, READ COMMITTED, REPEATABLE READ, SERIALIZABLE — listed from least to most restrictive. They control which concurrency anomalies (dirty reads, non-repeatable reads, phantom reads) are allowed.

Q25. What is a deadlock? How do you prevent it?

Two transactions waiting for each other's locks. Prevention: acquire locks in a consistent order, keep transactions short, use lower isolation levels when safe, and use timeouts/retry logic.

Q26. Difference between OPTIMISTIC and PESSIMISTIC locking?

Pessimistic: lock the row when reading. Optimistic: don't lock; check a version column at commit and fail if it changed. Optimistic is great for low-contention scenarios.

Q27. Difference between a stored procedure and a function?

Functions must return a value and can be used inside SELECT; they generally cannot perform DML/transactions. Procedures can have OUT params, perform DML/transactions, and are invoked with CALL.

Q28. What is a trigger? When would you use one?

A trigger is SQL that runs automatically on INSERT/UPDATE/DELETE. Use for audit logging, enforcing complex rules, and synchronizing data — but use sparingly to avoid hidden behavior.

Q29. What are window functions?

Functions that compute values across a set of rows (the 'window') without collapsing them like GROUP BY does. Examples: ROW_NUMBER, RANK, DENSE_RANK, LAG, LEAD, SUM/AVG OVER.

Q30. Difference between RANK, DENSE_RANK and ROW_NUMBER?

ROW_NUMBER: always sequential (1,2,3,4). RANK: skips after ties (1,1,3,4). DENSE_RANK: no gaps after ties (1,1,2,3).

Q31. What is a CTE? When is it preferred over a subquery?

Common Table Expression — a named temporary result set defined with WITH. Better for readability, reuse within the same query, and recursion.

Q32. What is a recursive CTE?

A CTE that references itself. Used for hierarchical data such as employee-manager trees, category trees, or computing paths.

Q33. How does indexing affect write performance?

Every INSERT/UPDATE/DELETE on indexed columns also updates the index, so heavy write workloads suffer if there are too many indexes.

Q34. What is the difference between EXPLAIN and EXPLAIN ANALYZE?

EXPLAIN shows the planner's expected query plan. EXPLAIN ANALYZE actually executes the query and shows real timings — useful for verifying performance.

Q35. Why is SELECT * considered a bad practice?

It fetches more data than needed, prevents covering-index usage, breaks if the schema changes, and increases network/IO load.

Q36. How would you find duplicate rows in a table?

Use GROUP BY on the columns that should be unique and HAVING COUNT(*) > 1.

Q37. How do you delete duplicates while keeping one copy?

Use ROW_NUMBER() OVER (PARTITION BY ... ORDER BY id) and delete rows where row_number > 1. Or use a self-join keeping MIN(id).

Q38. What's the difference between a primary key and a clustered index?

A primary key is a logical constraint. A clustered index is the physical storage order. In most RDBMS, the primary key automatically becomes the clustered index, but you can override that.

Q39. What is referential integrity?

The guarantee that foreign keys always reference valid rows in the parent table. Enforced via FK constraints and ON DELETE/UPDATE actions.

Q40. What is the N+1 query problem?

When fetching N parent records and then running 1 extra query per record to load related data, resulting in N+1 queries. Fix with JOINS or eager fetching (e.g., JPA @EntityGraph, JOIN FETCH).

Q41. What is SQL injection? How to prevent it?

An attack where user input is interpreted as SQL. Prevent by using PreparedStatement (parameterized queries), input validation, ORM, least-privilege DB accounts, and stored procedures.

Q42. Difference between TRUNCATE and DELETE for auto-increment?

TRUNCATE resets the AUTO_INCREMENT counter to its seed value. DELETE does not.

Q43. What is a covering index?

An index that contains all columns needed by a query, so the database can answer the query from the index alone (no table lookup).

Q44. Difference between OLTP and OLAP?

OLTP (Online Transaction Processing): many small fast transactions (banking, e-commerce).
OLAP (Online Analytical Processing): complex analytical queries over large datasets (BI, reporting).

Q45. How do you handle pagination efficiently in SQL?

Use LIMIT n OFFSET m for small datasets; for large datasets, use keyset pagination (WHERE id > last_seen_id ORDER BY id LIMIT n) which avoids the O(N) cost of OFFSET.

Q46. Why is using functions on WHERE clause columns bad?

It prevents the optimizer from using the index. Example: WHERE YEAR(hire_date) = 2024 is bad; use WHERE hire_date BETWEEN '2024-01-01' AND '2024-12-31' instead.

Q47. Difference between INNER JOIN and CROSS JOIN?

INNER JOIN returns matching rows based on a condition. CROSS JOIN returns the Cartesian product — every combination of rows.

Q48. What is the difference between scalar and aggregate functions?

Scalar: operate on a single value and return one (UPPER, LENGTH). Aggregate: operate on multiple values and return one summary (SUM, AVG).

Q49. Can you have a primary key with NULL? Why not?

No. The primary key uniquely identifies each row; NULL means 'unknown' which can't uniquely identify anything.

Q50. What is the difference between IS NULL and = NULL?

= NULL always returns UNKNOWN. IS NULL is a special predicate that returns TRUE when the value is NULL.

Q51. How do you copy a table with and without data?

With data: `CREATE TABLE new_t AS SELECT * FROM old_t`; Without data: `CREATE TABLE new_t LIKE old_t`; (MySQL) or `CREATE TABLE new_t AS SELECT * FROM old_t WHERE 1=0`; (generic).

Q52. Difference between JDBC Statement, PreparedStatement, and CallableStatement?

Statement: static SQL, parsed every time, vulnerable to SQL injection. PreparedStatement: parameterized, parsed once, safe. CallableStatement: invokes stored procedures with IN/OUT params.

Q53. What is connection pooling?

A cache of database connections reused across requests to avoid the overhead of opening new ones. Libraries: HikariCP (default in Spring Boot), Apache DBCP, c3p0.

Q54. What is lazy vs eager loading in JPA?

Lazy: related entities are loaded only when accessed. Eager: loaded immediately with the parent. Lazy avoids unnecessary loads but can cause N+1 if not handled.

Q55. What is the difference between save() and persist() in Hibernate?

save() returns the generated identifier and can be used outside a transaction. persist() returns void, must be called inside a transaction, and is part of the JPA spec.

Q56. What is the difference between get() and load() in Hibernate?

get() hits the database immediately and returns null if not found. load() returns a proxy and throws `ObjectNotFoundException` when accessed if the entity doesn't exist.

Q57. Explain optimistic locking using @Version.

JPA increments a `@Version` column on each update. If the version when reading doesn't match the version at commit time, `OptimisticLockException` is thrown — preventing lost updates.

Q58. How do you write a query to find the Nth highest salary?

Use `DENSE_RANK` or a correlated subquery. e.g., `SELECT salary FROM (SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) AS r FROM employees) t WHERE r = N`;

Q59. What is the difference between COUNT(*), COUNT(1), and COUNT(column)?

`COUNT(*)` and `COUNT(1)` count all rows (same performance in modern optimizers). `COUNT(column)` counts only non-NULL values of that column.

Q60. What is sharding?

Horizontally splitting a large dataset across multiple database servers (shards) to scale beyond the limits of a single machine. Each shard holds a subset of the rows.

Q61. What is replication?

Copying data from one DB server (primary/master) to others (replicas/slaves) for read scaling, high availability, and backups.

Q62. What is a composite index? What is the leftmost-prefix rule?

An index on multiple columns (a, b, c). Queries can use the index only if they filter on a leading prefix of the indexed columns (a; a,b; a,b,c) — filtering only on b or c won't use the index.

24. Practice Problems (Coding Round)

Practice writing these queries — they appear frequently in coding rounds of Java full stack interviews.

P1. Second highest salary.

```
SELECT MAX(salary) AS second_highest
FROM employees
WHERE salary < (SELECT MAX(salary) FROM employees);
```

P2. Nth highest salary (using DENSE_RANK).

```
SELECT salary FROM (
    SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk
    FROM employees
) t WHERE rnk = N;
```

P3. Find duplicate emails.

```
SELECT email, COUNT(*) AS cnt
FROM users
GROUP BY email
HAVING COUNT(*) > 1;
```

P4. Delete duplicate rows, keep one.

```
DELETE e1 FROM employees e1
INNER JOIN employees e2
    ON e1.email = e2.email
    AND e1.emp_id > e2.emp_id;
```

P5. Employees earning more than their manager.

```
SELECT e.name AS employee, m.name AS manager
FROM employees e JOIN employees m ON e.manager_id = m.emp_id
WHERE e.salary > m.salary;
```

P6. Department with highest average salary.

```
SELECT dept_id, AVG(salary) AS avg_sal
FROM employees
GROUP BY dept_id
ORDER BY avg_sal DESC
LIMIT 1;
```

P7. Employees who joined in the last 6 months.

```
SELECT * FROM employees
WHERE hire_date >= DATE_SUB(CURDATE(), INTERVAL 6 MONTH);
```

P8. Department-wise highest paid employee.

```
SELECT name, dept_id, salary FROM (
    SELECT name, dept_id, salary,
        RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS rk
    FROM employees
) t WHERE rk = 1;
```

P9. Customers who placed no orders.

```
SELECT c.customer_id, c.name
FROM customers c LEFT JOIN orders o ON c.customer_id = o.customer_id
WHERE o.order_id IS NULL;
-- Alternative
SELECT customer_id, name FROM customers
WHERE customer_id NOT IN (SELECT DISTINCT customer_id FROM orders);
```

P10. Running total of sales by date.

```
SELECT order_date, amount,
       SUM(amount) OVER (ORDER BY order_date) AS running_total
FROM orders;
```

P11. Top 3 products per category by revenue.

```
SELECT category, product, revenue FROM (
  SELECT category, product, revenue,
         ROW_NUMBER() OVER (PARTITION BY category ORDER BY revenue DESC) AS rn
  FROM sales
) t WHERE rn <= 3;
```

P12. Number of employees per department, including departments with zero.

```
SELECT d.dept_id, d.dept_name, COUNT(e.emp_id) AS headcount
FROM departments d LEFT JOIN employees e ON d.dept_id = e.dept_id
GROUP BY d.dept_id, d.dept_name;
```

P13. Pivot — count of orders per status per customer.

```
SELECT customer_id,
       SUM(CASE WHEN status='PENDING' THEN 1 ELSE 0 END) AS pending,
       SUM(CASE WHEN status='SHIPPED' THEN 1 ELSE 0 END) AS shipped,
       SUM(CASE WHEN status='DELIVERED' THEN 1 ELSE 0 END) AS delivered
FROM orders GROUP BY customer_id;
```

P14. Find consecutive logins by the same user on the same day (gaps and islands).

```
SELECT user_id, login_date,
       login_date - INTERVAL ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY login_date) DAY AS grp
FROM logins;
```

P15. Recursive CTE — full employee hierarchy.

```
WITH RECURSIVE org AS (
  SELECT emp_id, name, manager_id, 1 AS lvl
  FROM employees WHERE manager_id IS NULL
  UNION ALL
  SELECT e.emp_id, e.name, e.manager_id, o.lvl + 1
  FROM employees e JOIN org o ON e.manager_id = o.emp_id
)
SELECT * FROM org ORDER BY lvl, emp_id;
```

25. Final Tips, Tricky Concepts and Cheat Sheet

Tricky Concepts that Trip Candidates

- NULL comparison: `= NULL` always fails — use `IS NULL`.
- `COUNT(column)` ignores NULLs; `COUNT(*)` doesn't.
- `WHERE` filters rows, `HAVING` filters groups.
- `DELETE` without `WHERE` clears the entire table — and is logged.
- `DROP`, `TRUNCATE` are DDL and auto-commit in most engines.
- `ORDER BY` happens after `SELECT` — you can use column aliases there.
- `GROUP BY` happens before `SELECT` — you cannot use aliases in `GROUP BY` (in most DBs).
- When using `JOINS`, NULL columns in the result indicate no match.
- Indexes don't help if you wrap the column in a function: `WHERE LOWER(name) = 'arun'`.
- `PreparedStatement` parameter indexes start at 1, not 0.

Interview-day Cheat Sheet

```
-- BASICS
SELECT col FROM t WHERE cond ORDER BY col LIMIT n OFFSET m;

-- JOINS
SELECT a.x, b.y FROM a INNER JOIN b ON a.id=b.a_id;
SELECT a.x, b.y FROM a LEFT JOIN b ON a.id=b.a_id;
SELECT a.x, b.y FROM a RIGHT JOIN b ON a.id=b.a_id;

-- GROUPING
SELECT col, COUNT(*) FROM t GROUP BY col HAVING COUNT(*) > 1;

-- SUBQUERY
SELECT * FROM t WHERE col IN (SELECT col FROM t2 WHERE ...);

-- EXISTS
SELECT * FROM t WHERE EXISTS (SELECT 1 FROM t2 WHERE t2.id = t.id);

-- WINDOW
SELECT col, RANK() OVER (PARTITION BY x ORDER BY y DESC) FROM t;

-- CTE
WITH cte AS (SELECT ... FROM ...) SELECT * FROM cte WHERE ...;

-- TRANSACTION
START TRANSACTION; ...; COMMIT; -- or ROLLBACK;

-- INDEX
CREATE INDEX idx_name ON t(col);
```

Common Java + SQL Pitfalls

- Always use `PreparedStatement` — avoid concatenating user input into SQL.
- Close `ResultSet/Statement/Connection` (use `try-with-resources`).
- Don't forget to set `autoCommit(false)` for multi-statement transactions.

- Beware the JPA N+1 problem — use JOIN FETCH or @EntityGraph for collections.
- Validate that DB column types match Java types (DATE vs LocalDate, NUMERIC vs BigDecimal).
- Tune HikariCP pool size — default of 10 is usually enough; bigger isn't always better.

Last-minute Prep Checklist

- Be able to write any join in under 30 seconds.
- Know all 4 ACID properties by heart.
- Know the 4 isolation levels and which anomalies they prevent.
- Be ready to write 'second highest salary' in three different ways.
- Be able to explain Primary vs Unique vs Foreign vs Composite key.
- Be able to explain how indexes work at a high level (B-tree).
- Have one practical example for each: view, trigger, stored procedure, CTE.
- Have a JDBC snippet memorized end-to-end.
- Know what SQL injection is and how PreparedStatement prevents it.
- Practice 10–15 LeetCode SQL easy/medium problems before the interview.

All the best for your interviews!

Master the fundamentals, practice writing queries by hand on paper or a whiteboard, and always explain your thought process aloud while coding. Interviewers care as much about how you think as about the final answer. You've got this!